

Generated Drifts

X and a hand-designed
Randers field ω
`run_swiss_roll.py`
`/ run_mammoth.py:`
`make_*_randers()`

inject ω into edge weights,
knn adjacency, di-
rected Dijkstra $\rightarrow$ dense
 D_{asym} (all entries finite)
`randers_bridge.py:`
`compute_dist_matrix()` L60,
adjacency L148/175, ω
inject L213, Dijkstra L222

SECOND, independent
call with $\omega = \text{None}$ $\rightarrow$
 D_{geo} (own full Dijkstra)
`randers_bridge.py:` L544--545

“locate” graph: $X_{aug} =$
 $[X; X + \omega]$, symmetric
distances $\rightarrow$ classical_mds
`randers_bridge.py:`
`run_located_drift()` L469--504

split $Y_{aug0} \rightarrow Y_{real0}, Y_{virtual0}$
 $B_{located} = Y_{virtual0} -$
 Y_{real0} , FROZEN
 $Y_{init} = Y_{real0}$
`randers_bridge.py:` L506--509
($B_{located}$), L571--578 (passed
as B_{fixed} , $Y_{init_override}$)

Calculated Drifts

X alone – no field
injected anywhere

each point’s own true
directed k nearest neigh-
bors only $\rightarrow$ SPARSE
 D_{asym} (rest inf, NOT
Dijkstra-completed)
`isumap_bridge.py:`
`build_isumap_dist_matrix()` L31

D_{geo} derived ALGE-
BRAICALLY from D_{asym}
itself: average if both dir.
finite, else the finite one
`run_mammoth.isumap.py`
etc.: L123--125

Dijkstra-COMPLETE a
copy of D_{asym} (diameter-
fill unreachable pairs) $\rightarrow$
classical_mds $\rightarrow Y_{init}$
`isumap_bridge.py:`
`isumap_style_init()` L65

$N = _compute_N(D_{asym})$:
per-pair asymmetry signal,
diameter-substitution
fix for one-directional
pairs; $B_{fixed} = \text{None}$
`randers_umap.py:` `\_compute\_N()`
L330, called L951

randers_umap_fit() – Internal Pipeline
(shared code, runs identically for both families from here on)

build directed k -NN membership: A , $knn_mask = _knn_weights(D_{asym}, k)$
`randers_umap.py:` L907, `\_knn_weights()` L149

(optional) **force_edges** override: $\text{force } A[i, j] =$
1 for guaranteed edges, regardless of k -NN rank
`randers_umap.py:` L908--914

$(a, b_{param}) = \text{find_ab_params}(\text{spread}, \text{min_dist})$; $N =$
 $_compute_N(D_{asym})$ (once, fixed for the whole run)
`randers_umap.py:` L950--951, `find_ab_params()` L198, `\_compute\_N()` L330

$\text{drift_mask} = knn_mask$ ($\text{drift_mode} = \text{“knn”}$, the adopted default)
`randers_umap.py:` L953--958

if $\text{force_model} = \text{“fr_gravity”}$: $A_{edges} = knn_mask$
 $\vee knn_mask^T$ (undirected union); $k_{nat} = \text{fr_k}$ or $1/\sqrt{n}$
`randers_umap.py:` L966--969

$Y = Y_{init_override}$ if given, else $\text{classical_mds}(D_{asym}, d)$; $Y_{init} = Y.\text{copy}()$
`randers_umap.py:` L971--979, `classical_mds()` L218

$B = \mathbf{0}$ if B_{fixed} is None else $B_{fixed}.\text{copy}()$; $M = \text{node_mass}$ or all-ones
`randers_umap.py:` L981--982

----- *EPOCH LOOP: repeats n_epochs times* -----

ramp schedule: $s = 0$ (first 30% of epochs) $\rightarrow$ linear 0 $\rightarrow$ 1
(30–70%) $\rightarrow s = 1$ (last 30%); $s = 1$ always if $\text{ramp} = \text{False}$
`randers_umap.py:` L1074--1078

recompute pairwise geometry from the CURRENT Y : diff, d_{mat} , unit vectors e
`randers_umap.py:` L1085--1087

B_{fixed} given $\rightarrow B = s \cdot B_{fixed}$ (frozen direction;
optional live k -NN-distance magnitude rescale)
`randers_umap.py:` L1089--1123

$\text{use_drift} = \text{True}$, $B_{fixed} = \text{None} \rightarrow B =$
 $s \cdot \text{compute_drift}.\text{scaled}(N, \text{drift_mask},$
 $k, Y, \text{clip}.\text{delta})$
`randers_umap.py:` L1124--1126, `compute_drift()`
L390, `\_compute_drift_scaled()` L414

build (ρ_r, g_r) Randers-perturbed and (ρ_e, g_e) Euclidean pairs; select
 $\rho_{attr}/g_{attr}, \rho_{rep}/g_{rep}$ per `randers_attractive`/`randers_repulsive`
`randers_umap.py:` L1132--1139

$\text{force_model} = \text{“umap”}$: UMAP (a, b) -curve
 $\text{attr_coeff}/\text{rep_coeff}$, weighted by A_{attr}/A_{rep}
`randers_umap.py:` L1143--1165

$\text{force_model} = \text{“fr_gravity”}$: Bannis-
ter et al.: $f_a = \rho/k_{nat}$ (edges only,
 A_{edges}), $f_r = k_{nat}^2/\rho^2$ (all pairs)
`randers_umap.py:` L1166--1182

$\text{step} = \text{force.sum}(\text{axis}=1)$ – net force on each y_i
`randers_umap.py:` L1183--1188

OPTIONAL, additive: if **negative_sampling**: draw S fresh random negatives
per node this epoch, replace/augment repulsion (formula differs by force_model)
`randers_umap.py:` L1219--1244

OPTIONAL, additive: if **use_gravity**: $\text{step} += \gamma M w_{grav} \cdot \text{gravity_term}$
(pulls $y_i \rightarrow y_i + b_i$; plain linear or Randers pseudo-edge curve)
`randers_umap.py:` L1253--1291

OPTIONAL, additive: if **use_virtual_neighbor**: step
 $+= \text{attr}.\text{coeff}_v \cdot B$ (unconditional $(k+1)$ -th neighbour pull)
`randers_umap.py:` L1299--1303

clip step to $[-\text{grad_clip}, \text{grad_clip}]$; decaying lr $\text{cur_lr} =$
 $\text{lr} \cdot (1 - \text{epoch}/n_{epochs})$; $Y += \text{cur_lr} \cdot \text{step}$
`randers_umap.py:` L1305--1309

----- *end of epoch loop, next epoch (or exit after n_epochs)* -----

final B consistency recompute: live drift recomputed once more against the
FINAL Y ; or B_{fixed} magnitude rescaled against the final Y ’s k -NN distance
`randers_umap.py:` L1322--1342

return $\{Y, Y_{init}, B, \mu, \mu_{\text{directed}}, knn_mask, N, a, b_{param}, \text{history}\}$
`randers_umap.py:` L1350--1362